

Reinforcement Learning for AI as a Service: CPU-GPU Task Scheduling for Preprocessing, Training, and Inference Tasks

Ying-Dar Lin, *Fellow, IEEE*, Yin-Tao Ling[✉], Yuan-Cheng Lai[✉], and Didik Sudyana[✉]

Abstract—The rise of AI solutions has driven the emergence of AI as a Service (AIaaS), offering cost-effective and scalable solutions by outsourcing AI functionalities to specialized providers. Within AIaaS, three key components are essential: segmenting AI services into preprocessing, training, and inference tasks; utilizing GPU-CPU heterogeneous systems where GPUs handle parallel processing and CPUs manage sequential tasks; and minimizing latency in a distributed architecture consisting of cloud, edge, and fog computing. Efficient task scheduling is crucial to optimize performance across these components. In order to enhance task scheduling in AIaaS, we propose a user-experience-and-performance-balanced reinforcement learning (UXP-RL) algorithm. The UXP-RL algorithm considers 11 factors, including queuing task information. It then estimates resource release times and observes previous action outcomes, to select the optimal AI task for execution on either a GPU or CPU. This method effectively reduces the average turnaround time, particularly for rapid inference tasks. Our experimental findings show that the proposed RL-based scheduling algorithm reduces average turnaround time by 27.66% to 57.81% compared to the heuristic approaches such as SJF and FCFS. In a distributed architecture, utilizing distributed RL schedulers reduces the average turnaround time by 89.07% compared to a centralized scheduler.

Index Terms—AI as a service, CPU, GPU, task scheduling, reinforcement learning, deep Q-learning.

I. INTRODUCTION

THE WIDESPREAD adoption of AI solutions across applications has resulted in a rapid increase in users, which in turn has presented challenges related to compatibility, integration, and resource management. In response to these challenges, the concept of AI as a Service (AIaaS) has emerged [1]. By outsourcing AI functionalities to specialized providers, this trend offers cost-effective and scalable solutions for subscribers.

Received 18 September 2024; revised 24 February 2025; accepted 20 April 2025. Date of publication 25 April 2025; date of current version 7 August 2025. The associate editor coordinating the review of this article and approving it for publication was N. Kumar. (*Corresponding author: Didik Sudyana.*)

Ying-Dar Lin and Yin-Tao Ling are with the Department of Computer Science, National Yang Ming Chiao Tung University, Hsinchu 300, Taiwan (e-mail: ydlin@cs.nycu.edu.tw).

Yuan-Cheng Lai is with the Department of Information Management, National Taiwan University of Science and Technology, Taipei 106, Taiwan (e-mail: laiy@cs.ntust.edu.tw).

Didik Sudyana is with the Computer and Network Center, National Cheng Kung University, Tainan 701, Taiwan (e-mail: didik@gs.ncku.edu.tw).

Digital Object Identifier 10.1109/TNSM.2025.3564480

Efficiently managing numerous AI services is essential for AIaaS providers. This involves segmenting services into individual AI tasks across preprocessing, training, and inference stages [2]. Each task has distinct requirements: preprocessing involves sequential data input for training preparation, training entails iterative matrix calculations and backpropagation updates, and inference demands rapid turnaround for single predictions. These tasks showcase different characteristics that must be considered.

In typical cluster infrastructures, GPU-CPU heterogeneous systems are common. GPUs are optimized for parallel processing tasks, such as graphics rendering and deep learning training, due to their large number of cores. Conversely, CPUs excel at handling sequential tasks. By combining both GPU and CPU resources, heterogeneous systems can efficiently handle diverse workloads, making them valuable for AI tasks.

Optimizing performance in AIaaS relies on effective task scheduling, balancing GPU parallel processing and CPU sequential processing across various AI tasks [3]. This involves making sequential decisions to allocate tasks to available computational resources. The diverse characteristics of AI tasks on heterogeneous resources add complexity to scheduling, requiring advanced algorithms for effective management.

Task scheduling is also critical in distributed AIaaS architectures, which transition from centralized architectures with only a server site to distributed architectures across multiple tiers [4]. These tiers, including Cloud, Edge, and Fog computing, aim to minimize latency by bringing computation closer to users. While the Cloud offers high capacity, it often faces latency issues that Edge and Fog computing can mitigate despite their limited capacity. Distributed architectures introduce challenges in synchronizing sub-cluster information and load balancing. Thus, optimizing task scheduling requires understanding AI task requirements and resource availability across the distributed network.

Numerous studies on AI task scheduling have been proposed, primarily focusing on specific tasks such as training [5], [6], [7], [8], [9], [10], [11] or inference [12], [13], [14], [15], [16]. Few have considered diverse AI tasks [17]. While some studies explore both CPU and GPU resources [5], [7], [11], [14], [15], [16], [17], many focus solely on GPUs [5], [6], [7], [8], [9], [10], [11], [12], [17]. Leveraging diverse resource combinations without predefined requirements can optimize task scheduling better.

Most foundational studies focus on centralized architectures, whereas distributed architectures leverage fog, edge, and cloud computing to offer lower transmission latency, which can benefit AIaaS environments. Additionally, many existing approaches rely on heuristic [12], [17] and machine learning (ML)-based [5], [6], [7], [14] methods. One of the key challenges with ML-based approaches, particularly supervised learning, is the requirement for a well-labeled dataset before training the model. Constructing such a dataset in a highly dynamic scheduling environment is both costly and impractical. AIaaS platforms often deal with fluctuating workloads, varying resource availability, and continuously changing network conditions, making it difficult to obtain a static, representative dataset for supervised learning. Even if a dataset could be created, frequent updates would be necessary to maintain accuracy, increasing maintenance costs. Moreover, some AIaaS providers may restrict access to detailed operational data due to security and privacy concerns, further limiting the feasibility of supervised ML approaches.

In contrast, unsupervised ML methods, such as clustering or anomaly detection, do not require labeled data, making them more flexible than supervised learning in dynamic environments. However, these approaches face challenges in scheduling optimization because they lack a clear objective-driven learning mechanism. Unsupervised methods can group similar tasks or detect scheduling inefficiencies, but they do not inherently optimize scheduling decisions in real time.

To address these limitations, we leverage RL for task scheduling optimization. RL learns optimal scheduling policies dynamically through interactions with the environment. It continuously adapts to changing workloads, resource availability, and system constraints, refining its decision-making strategies over time. Unlike static ML models, RL balances exploration and exploitation, discovering new, efficient scheduling strategies without requiring extensive offline training.

Additionally, RL enables forecasting of future outcomes based on past actions, making it highly effective for managing heterogeneous computing resources such as CPUs and GPUs. This predictive capability ensures that resources are allocated efficiently, accounting for both immediate and long-term impacts of scheduling decisions.

By leveraging RL's strengths in real-time learning and adaptability, we address critical challenges in task scheduling, such as minimizing turnaround time (TAT), balancing workloads across heterogeneous resources, and scaling to accommodate diverse AI tasks. These factors make RL an ideal choice for developing our scheduling optimization algorithm.

Building on these capabilities, we propose a user-experience-and-performance-balanced RL (UXP-RL) algorithm for AIaaS task scheduling, optimizing both user experience and system performance. It reduces turnaround time by efficiently allocating tasks to GPUs or CPUs while minimizing latency, especially for inference tasks. By considering queuing information, estimated resource release times, and past performance, the UXP-RL precisely manages resources. We evaluate it using AI task execution times under CPU-GPU constraints and different scheduling intervals, demonstrating improvements over heuristic algorithms in a simulated environment.

To illustrate the practicality of our approach, an AI-powered video analytics application for real-time traffic monitoring in smart cities serves as a pertinent example. This application encompasses a range of AI tasks: preprocessing video streams from edge devices, training models to detect traffic patterns, and executing inference tasks for real-time detection of vehicles and pedestrians. These tasks are categorized under preprocessing, training, and inference, each demanding specific resource allocations and subject to stringent latency constraints.

The RL-based scheduling algorithm optimizes the deployment of these tasks across heterogeneous computing environments—cloud, edge, and fog. By dynamically managing resources, the algorithm ensures a balance between GPU and CPU workloads, tailoring configurations to meet the system's real-time requirements, and reducing latency. This adaptive resource management leads to significant reductions in turnaround time, enhances scalability, and boosts overall performance, particularly critical for the demands of real-time analytics.

In this work, we explore two approaches for task scheduling: the centralized and distributed models. The centralized model uses a single cloud-based scheduler with a global view, simplifying resource allocation but risking bottlenecks and increased latency. The distributed model, by contrast, spreads scheduling across multiple nodes, allowing for localized, real-time decisions that reduce latency and improve scalability, though it requires more complex coordination.

Our work contributes in three key ways:

- *Comprehensive Task Scheduling:* We address the task scheduling problem in AIaaS by considering three critical factors: AI tasks, CPU-GPU resources, and distributed architectures. The complexity we tackle exceeds that of existing studies.
- *Adaptive RL Task Scheduling Algorithms:* We propose an adaptive RL algorithm named UXP-RL, which enables service providers to optimize performance and user experience based on the preferences of both providers and users.
- *Performance Comparison and Trade-off Analysis:* We compare the performance of our UXP-RL and heuristic approaches in terms of AI task execution times across different scheduling intervals, providing numerical trade-off results.

The rest of this paper is organized as follows: in Section II, we provide the background and related works. Sections III and VI present the problem statement and solution approaches, respectively. In Section V, we describe the detailed implementation. The experiment results are shown in Section VI. Finally, Section VII concludes the works and discusses future works.

II. BACKGROUND AND RELATED WORK

A. AI as a Service

With the rising demand for machine learning, third-party providers offer AIaaS to manage diverse AI services efficiently. The AIaaS architecture, as shown in Figure 1, consists of multiple interconnected components designed to handle

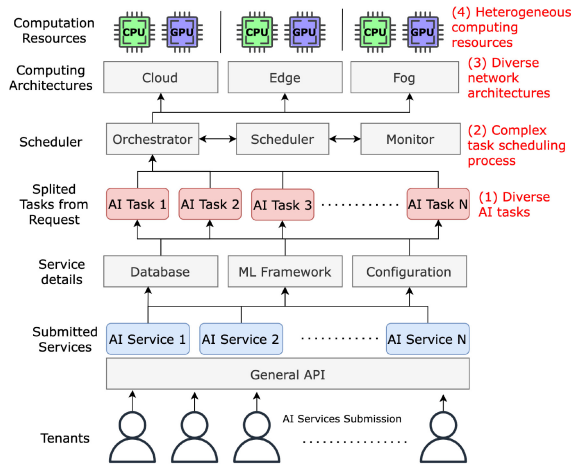


Fig. 1. Architecture of AIaaS.

tenant-submitted AI services. Tenants register their service details, including resource requirements, application codes, and datasets, which are then translated into diverse AI tasks. These tasks vary in complexity and are split into manageable units for efficient deployment.

At the core of the system, an orchestrator collaborates with a scheduler to allocate tasks across computing architectures—cloud, edge, and fog environments—while considering the availability of heterogeneous resources, such as GPUs and CPUs. A monitoring module continuously collects cluster performance data to support real-time optimization and ensure system reliability. The scheduler plays a critical role in mapping resources and selecting appropriate configurations for machine learning frameworks. This process must account for task size, complexity, and the underlying network architecture to optimize runtime performance and maximize resource utilization. Modern AI applications often operate in heterogeneous environments, leveraging both cloud-scale resources and low-latency edge and fog setups. As a result, the architecture must manage diverse network architectures, balance workloads effectively, and ensure scalability to handle the growing number of AI services.

B. GPU Acceleration for AI Tasks

Since the 2010s, GPU utilization for accelerating calculations has surged beyond graphics, offering significant enhancements in AI development through parallel execution. However, not all AI workflows benefit equally from GPUs. Table I compares execution times between AI tasks on GPU and CPU, considering input data in minibatch size. Preprocessing raw data shows minimal differences between CPU and GPU [18]. However, GPUs significantly enhance model calculations. For example, in VGG-19 [19], GPUs are 47x faster for training and 52x for inference; in ResNet50 [20], they are 44x and 49x faster, respectively. These variations are due to differences in backpropagation mechanisms, underscoring the potential to optimize AI tasks with GPU-CPU resources.

TABLE I
PERFORMANCE COMPARISON OF AI TASKS ON CPU AND GPU

AI Task	Model	CPU Time (ms)	GPU Time (ms)	GPU Speedup
Preprocessing	-	0.52	0.54	0.95x
Training	VGG19	9849.23	210.39	47x
	Resnet50	6627.25	149.82	44x
Inference	VGG19	3609.78	68.95	52x
	Resnet50	2477.61	50.64	49x

- Benchmark with a minibatch size of 16 and with size 224 x 224, ImageNet
- Time = Turnaround time for 16 samples
- CPU: Dual Xeon E5-2630 v3, GPU: GTX 1080 8GB

C. Reinforcement Learning

RL consists of three core components: environment, agent, and reward. Agents explore the environment to maximize cumulative rewards and learning through continuous or discrete actions. Rewards provide immediate feedback, guiding decisions to optimize policies and maximize rewards across all states. RL algorithms are categorized into model-based and model-free approaches [21]. Model-based RL involves explicit environment modeling, while model-free RL learns policies without it. Due to AIaaS complexity, we prefer model-free methods such as policy-based, value-based, and critic-actor RL. Policy-based RL learns optimal policies directly, value-based RL estimates action values in states, and critic-actor RL combines value estimation with policy improvement. In task scheduling, policy-based and critic-actor RL methods are redundant due to task specificity.

Value-based methods such as Deep Q-Network (DQN) [22], Double Deep Q-Network (DDQN) [23], and Categorical DQN (C51) [24] can efficiently find optimal solutions. DQN enables agents to learn optimal actions by approximating the Q-value function, while DDQN uses two separate networks for action selection and evaluation. C51 enhances DQN by representing value returns with value distribution instead of expected values. However, due to the complexity and resource demands of DDQN and C51, we prefer the simplicity and effectiveness of DQN for task scheduling in AIaaS environments.

D. Related Works

Table II highlights the differences between prior research and our study, focusing on two aspects: problem and solution. The problem section evaluates AI tasks, computational resources, and network architecture. The solution section outlines the primary goal, workload prediction, and scheduling algorithm approaches.

Previous studies have mainly focused on specific AI tasks such as training [5], [6], [7], [8], [9], [10], [11] and inference [12], [13], [14], [15], [16], with only one addressing multi-tasks [17]. Training tasks take days to months, allowing ample time for scheduling decisions, while inference tasks must be completed quickly, often facing long queuing delays. Our goal is to devise a flexible solution that incorporates pre-processing, training, and inference tasks for a comprehensive approach.

Computational resources are crucial for task scheduling. While most works rely on GPU acceleration, some studies [5], [7], [11], [14], [15], [16], [17] have explored using both

TABLE II
SURVEY ON TASK SCHEDULING FOR DEEP LEARNING SERVICES

Papers	AI Tasks	Computation Units			Architecture	Objective	Approaches	
		CPU	GPU	CPU-GPU Selection			Workload Prediction	Scheduling Algorithm
[5]	T	✓	✓	✗	C	Throughput	DL	Hybrid heuristic
[6]		✗	✓	✗	C	Cost	DL	MILP
[7]		✓	✓	✗	C	Turnaround time	DL	RL
[8]		✗	✓	✗	C	Turnaround time	Heuristic	Heuristic
[9]		✗	✓	✗	C	Cost	Historical	Heuristic
[10]		✗	✓	✗	C	Turnaround time	Historical	RL
[11]		✓	✓	✗	C	Execution time	Historical	Heuristic
[12]	I	✗	✓	✗	C	Turnaround time	Historical	Hybrid heuristic
[13]		✓	✗	✗	E	Execution time	Historical	Heuristic
[14]		✓	✓	✓	E	Cost	Heuristic	RL
[15]		✓	✓	✓	C	Turnaround time	DL	DL
[16]		✓	✓	✓	C	Cost	RL	RL
[17]		✓	✓	✗	C	queuing time	Historical	Heuristic
Ours (UXP-RL)	P / T / I	✓	✓	✓	C/E/F	Turnaround time	RL	RL

P/T/I: Preprocessing/Training/Inference, C/E/F: Cloud/Edge/Fog, DL/RL: Deep Learning/Reinforcement Learning

CPU and GPU resources. However, they often treat resource requirements as static and predefined criteria, which limits their adaptability to dynamic workloads. For instance, the works in [14] and [15] propose scheduling methods that leverage both CPU and GPU resources, focusing on minimizing costs and turnaround time, respectively. However, neither of these studies considers the complexities of distributed architectures spanning cloud, edge, and fog computing. Additionally, they overlook the diverse requirements of different AI tasks, such as preprocessing, training, and inference, instead focusing on a narrow scope of task types and centralized resource allocation strategies. Similar to [16], while exploring CPU-GPU selection to reduce latency and optimize communication, it assumes relatively static resource environments and does not account for dynamic resource allocation or the variability of AI tasks, which are central to AIaaS platforms. These limitations restrict their applicability in dynamic AIaaS environments where workload variability and resource distribution are critical.

In contrast, our proposed approach introduces dynamic CPU-GPU selection by allocating tasks to either CPU or GPU based on real-time availability and task-specific requirements. This dynamic allocation not only optimizes performance and reduces latency but also adapts to the unique characteristics of preprocessing, training, and inference tasks. Furthermore, unlike the aforementioned works, our approach is designed for distributed architectures across cloud, edge, and fog tiers, enabling localized decision-making and improved scalability. By addressing these challenges, our method balances system performance and user experience, making it well-suited for modern AIaaS environments.

In the objectives section, turnaround time refers to the duration from task initiation to completion. Considering all task phases allows for optimization at every stage. While [11] focuses on task execution feedback and [17] addresses resource allocation before scheduling, neither directly impacts execution time. We evaluate the situation more broadly.

In the solution section, existing literature offers algorithmic approaches relying on pre-run prediction procedures to obtain estimated metrics, using historical data [9], [10], [11], [12], [17], DL-based methods [5], [6], [7], and heuristics [8]. These

methods require efforts to develop prediction models from historical data. We propose using parameters from AI task requests without additional processing.

Our work addresses four key points not covered by previous studies:

- *Comprehensive Task Scheduling:* We consider all three AI tasks, tackling short decision times and varying execution durations.
- *Flexible Resource Selection:* We soften filtering with computational resource (CPU-GPU) selection, enabling flexible scheduling when one resource is depleted.
- *Distributed Architecture Adaptation:* We adapt task scheduling methods for distributed architectures, enhancing scalability and flexibility.
- *RL-Based Approaches:* We propose RL-based adaptive methods named UXP-RL to reduce delays and optimize execution times without additional data preparation.

III. PROBLEM DESCRIPTION

This section addresses the problem statement through two parts: the system model and problem formulation. We introduce our cluster model for task scheduling, explaining its design choices. A notation table outlining the problem is included. Finally, we present the task scheduling problem with mathematical formulas, considering various metrics as input parameters and clarifying our objectives.

A. System Model

Table III lists all essential notations used in our study, organized into seven groups with columns for notation, description, and attributes. The first three categories cover network-related details in distributed systems. The next two categories define AI tasks and their key parameters. The scheduled resource category shows resource allocation outputs, and the final category includes timestamp notations.

In our study, each group of clustered nodes at each tier is referred to as a “data center,” comprising multiple CPU and GPU devices in a hierarchical structure. Each data center has its own scheduler, which can communicate with other schedulers by transmitting AI task requests between tiers.

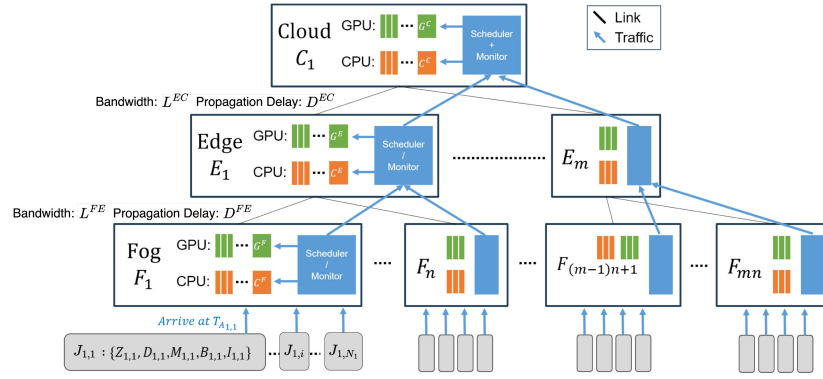


Fig. 2. Structure for task scheduling in AIaaS.

TABLE III
MODEL NOTATIONS

Category	Notation	Description	Attribute
Topology	m	Number of edge data centers	Input
	n	Number of fog data centers for each edge data center	Input
Number Of Resource	G^C	Number of GPU units at cloud data center	Input
	G^E	Number of GPU units at edge data center	Input
	G^F	Number of GPU units at fog data center	Input
	C^C	Number of CPU units at cloud data center	Input
	C^E	Number of CPU units at edge data center	Input
	C^F	Number of CPU units at fog data center	Input
Network	L^{FE}	Link bandwidth between fog and edge data centers	Input
	L^{EC}	Link bandwidth between edge and cloud data centers	Input
	D^{FE}	Propagation delay between fog and edge data centers	Input
	D^{EC}	Propagation delay between edge and cloud data centers	Input
AI Task	$J_{i,j}$	j -th arrival AI task at i -th fog data center where $1 \leq j \leq N_i$, N_i is the number of AI tasks arriving at the i -th fog $1 \leq i \leq m \cdot n$	Input
AI Parameter	$Z_{i,j}$	Type of j -th arrival AI task at i -th fog where $Z \in \{1, 2, 3\}$ as Preprocessing, Training and Inference	Input
	$D_{i,j}$	Number of dataset samples in j -th arrival AI task at i -th fog	Input
	$M_{i,j}$	Number of AI model size in j -th arrival AI task at i -th fog	Input
	$B_{i,j}$	Number of AI batch size in j -th arrival AI task at i -th fog	Input
	$I_{i,j}$	Number of AI processing iteration in j -th arrival AI task at i -th fog	Input
	r	Number of bytes representing AI model data type	Input
Scheduled Resource	s	Number of a dataset sample size	Input
	$G_{i,j}$	GPU for j -th arrival AI task at i -th fog	Output
Timestamp	$C_{i,j}$	CPU for j -th arrival AI task at i -th fog where $G_{i,j}, C_{i,j} \in \{1, 2, 3\}$ as cloud, edge and fog data center	Output
	$T_{A,i,j}$	Arrival time for j -th arrival AI task at i -th fog	Input
	$T_{S,i,j}$	Scheduled time for j -th arrival AI task at i -th fog	Output
Duration	$T_{D,i,j}$	Departure time for j -th arrival AI task at i -th fog	Output
	$U_{Q,i,j}$	queuing time for j -th arrival AI task at i -th fog	Output
	$U_{E,i,j}$	Execution time for j -th arrival AI task at i -th fog	Output
	$U_{T,i,j}$	Turnaround time for j -th arrival AI task at i -th fog	Output

Fig. 2 illustrates a distributed architecture comprising one cloud data center, m edge data centers, and n fog data centers per edge data center, based on ETSI's [25] and 5G-Coral frameworks [26]. In our model, the cloud and edge belong to the service provider's domain, while the fog is within the subscriber's domain. Edge computing aligns with the Multi-access Edge Computing (MEC) concept, referring to the edge of the service provider's network.

Each data center includes a monitor and a scheduler. The monitor collects data such as queue pool status, resource availability, and execution outcomes, updating in real-time with millisecond precision. The scheduler uses this data to allocate tasks from the queue to appropriate computational resources. Monitors update the current status independently.

Our focus is on scheduling, assuming all data centers have monitoring systems that can exchange information in real time. Schedulers can be deployed differently: centralized (one

scheduler in the cloud) or distributed (schedulers on each node). In a centralized system, non-scheduler data centers receive decisions from the main scheduler, while distributed schedulers can independently transmit AI task requests to upper tiers.

Offloading mechanisms allow a fog data center to pass a task request to an edge data center, shifting the scheduling responsibility upward. This vertical scheduling process, also applicable between edge and cloud data centers, is a common approach in distributed architectures [27].

Each data center possesses varying resource capacities based on its tier level. When considering resources in a distributed architecture, two primary factors come into play: computational resources and network resources.

- Computational resources include GPUs and CPUs, with performance normalized into factors G and C . Each data center in different tiers has varying GPU and CPU

capacities: G^F and C^F in the fog data center, G^E and C^E in the edge data center, and G^C and C^C in the cloud data center. We assume that either the CPU or GPU can handle only one task at a time and must complete that task before serving the next one. Resources are quantified by the number of virtual instances of physical resources, providing flexibility for AIaaS providers in managing availability.

- The bandwidth between fog and edge data centers, denoted as L^{FE} , and between edge and cloud data centers, denoted as L^{EC} , constitutes network resources. In addition to the network resource capacity, propagation latencies D^{FE} and D^{EC} can also be observed during data transmission between data centers.

Task scheduling occurs after services are divided into AI tasks, represented as $J_{i,j}$, the j -th task at the i -th fog data center. Each of the $m \cdot n$ fog data centers receives its own set of AI tasks. The size of the AI task set is N_i , indicating the total tasks at the i -th fog data center. Each j -th AI task, $J_{i,j}$, is categorized by AI type $Z_{i,j}$ (preprocessing, training, inference).

AI task request $J_{i,j}$ has configurations for the AIaaS execution environment, with input metrics including dataset size $D_{i,j}$, AI model size $M_{i,j}$, batch size $B_{i,j}$, and total processing iterations $I_{i,j}$. These metrics are crucial for evaluating performance on CPU and GPU platforms [28]. Preprocessing tasks do not require AI model size and batch size.

To calculate transmission latency after scheduling, each task is allocated specific computational resources across cloud, edge, and fog levels, denoted as $G_{i,j}$ and $C_{i,j}$ (1 for cloud, 2 for edge, 3 for fog). For example, a task assigned to a GPU in the edge data center is denoted as $G_{i,j} = 2$.

We present five timestamp notations for task scheduling stages. Each task has an arrival time at the fog data center, denoted as $T_{A_{i,j}}$. Upon receiving the request, the scheduler stores it in the task pool, with the duration spent in the queue referred to as queuing time ($U_{Q_{i,j}}$). After assigning the task to an available resource, the scheduled time is noted as $T_{S_{i,j}}$. The execution time within the assigned resources is $U_{E_{i,j}}$. Once the task is completed, the departure timestamp is recorded as $T_{D_{i,j}}$. The turnaround time ($U_{T_{i,j}}$) represents the total duration from task submission to completion and departure from the AIaaS platform.

Note that the current cluster systems do not support preemption. Preempting an AI task involves workers pausing and resuming tasks frequently. Checkpointed model files and datasets need to be migrated and stored across various memory scales, devices, and nodes, resulting in significant time overhead [29]. For this reason, we consider our cluster to be in a non-preemptive environment.

B. Problem Statement

The task scheduling problem in the AIaaS environment can be formulated as an optimization problem with the objective of minimizing turnaround time while adhering to resource constraints.

Given:

- Multi-tier computing architecture:
 - m edge data centers, each with n fog data centers.
 - Computing devices at different tiers:
 - * Cloud: G^C, C^C (GPU and CPU resources).
 - * Edge: G^E, C^E .
 - * Fog: G^F, C^F .
- Incoming AI tasks $J_{i,j}$, each defined by:
 - Arrival time: $T_{A_{i,j}}$.
 - Task size parameters: Number of samples $D_{i,j}$, model size $M_{i,j}$, batch size $B_{i,j}$, and iteration count $I_{i,j}$.
- Network constraints:
 - Bandwidth between Fog-Edge L^{FE} and Edge-Cloud L^{EC} .
 - Propagation delay between Fog-Edge D^{FE} and Edge-Cloud D^{EC} .

Output:

- Resource type and location: Assigning the task to either a GPU or CPU in a Cloud, Edge, or Fog node:

$$G_{i,j} \vee C_{i,j} \quad (1)$$

- Scheduling time: Start time $T_{S_{i,j}}$ and completion time $T_{D_{i,j}}$.
- Execution duration:

$$U_{Q_{i,j}}, U_{E_{i,j}}, U_{T_{i,j}} \quad (2)$$

Objective:

- Minimize turnaround time U_T , which consists of queuing time, execution time, and communication delays:

$$\min \sum_{i,j} U_{T_{i,j}} = \sum_{i,j} (U_{Q_{i,j}} + U_{E_{i,j}} + D_{i,j}) \quad (3)$$

where $D_{i,j}$ is the communication delay incurred in scheduling the task.

Constraints:

- Resource Exclusivity: A task can either be assigned to a GPU or a CPU but not both:

$$G_{i,j} \neq 0 \Rightarrow C_{i,j} = 0, \quad C_{i,j} \neq 0 \Rightarrow G_{i,j} = 0 \quad (4)$$

- Resource Availability: The sum of allocated resources must not exceed available resources at any given time:

$$\sum_{i,j} G_{i,j} \leq G^X, \quad \sum_{i,j} C_{i,j} \leq C^X, \quad X \in \{C, E, F\} \quad (5)$$

- Scheduling Constraint: The start time of a task must be after its arrival time:

$$T_{S_{i,j}} \geq T_{A_{i,j}} \quad (6)$$

IV. UXP-RL: USER EXPERIENCE AND PERFORMANCE BALANCED REINFORCEMENT LEARNING

This section begins with an overview of the solutions; then we discuss the details of the UXP-RL solution in the centralized and distributed schedulers.

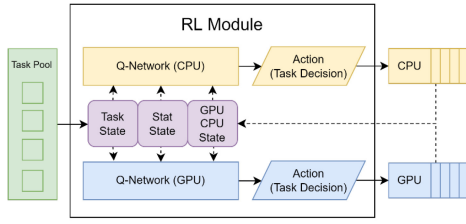


Fig. 3. Components in centralized RL scheduler.

A. Solution Overview

For our task scheduling problem in AIaaS, we examine two approaches of UXP-RL: centralized and distributed RL scheduler models. In the centralized model, a single scheduler in the cloud data center makes all decisions, offering easy implementation and a comprehensive system view but risking overhead and bottlenecks. In contrast, the distributed model delegates scheduling decisions to multiple nodes, allowing localized decision-making based on local resources and requirements. This distributed approach, built upon the centralized solution, offers greater adaptability and flexibility, avoiding the latency issues of a single central point.

B. Centralized RL Scheduler

As the central control point, all requests must be submitted to the scheduler in the cloud data center, which maintains a global map of AI tasks and resources. This scheduler allocates AI tasks ($J_{i,j}$) and computational resources ($G_{i,j}$ and $C_{i,j}$). However, the combination of tasks and resources creates an exponentially large decision space.

To reduce complexity, we transform the scheduler into an event-triggered system, aiming to decrease computation time. Event triggers occur when a task is submitted and when it is finished. By ordering these events, the current task queue forms a sequence suitable for RL. With each trigger, RL observes the queue and selects the next task to schedule. Monitors provide feedback only at these triggers to avoid overloading the network and preserve bandwidth.

We propose deploying an RL module in a central scheduler, as depicted in Fig. 3. Algorithm 1 outlines the details of the central scheduler. In our algorithm, we choose the DQN [22] as our RL algorithm. Given that our task scheduling only involves discrete decision choices, its value-based characteristics can efficiently find optimal solutions swiftly. Simpler RL models are advantageous for our scheduler deployment, hence leading us to utilize the DQN algorithm.

Deciding on resources prior to task scheduling involves separating the two types of computational resources, G (GPU) and C (CPU), into distinct decision-making processes, with one agent assigned to each resource type. Specifically, we use a two-agent structure, where one agent manages GPU scheduling and the other handles CPU scheduling. Each agent acts as a task scheduler with the help of the monitor component in AIaaS, providing state information for RL. This structure enhances scalability and reduces training convergence time by allowing the agents to operate independently based on resource

Algorithm 1 UXP-RL Centralized RL Scheduler

Input: Replay memory D^G, D^C with memory size N
Input: Q-network with random weights θ^G, θ^C
Input: Target Q-network $\theta_{\text{target}}^G = \theta^G, \theta_{\text{target}}^C = \theta^C$
Input: exploration rate ϵ , decay rate ρ , discount factor γ ,
Input: learning rate α , minibatch size B , update frequency I

```

1: for episode = 1 to  $M$  do
2:   Initialize state  $s_1^G, s_1^C$ 
3:   Initialize total reward  $R = 0$ 
4:   for  $t = 1$  to  $T_{\text{max}}$  do
5:     for agent  $k = \{G, C\}$  do
6:       if  $t$  is not event triggered then
7:         continue
8:       end if
9:       Get  $s_t^k = (s_t^{\text{task}}, s_t^{\text{resource}}, s_t^{\text{stat}})$  from monitors
10:      Select  $J_{i,j}$  from  $a_t^k = \arg \max_a Q(s_t^k, a^k; \theta^k)$  using  $\epsilon$ -greedy policy
11:      Schedule  $J_{i,j}$  to get reward  $r_t^k$  and next state  $s_{t+1}^k$ 
12:      Store transition  $(s_t^k, a_t^k, r_t^k, s_{t+1}^k)$  in replay memory  $D^k$ 
13:      Sample a random minibatch of transitions  $(s_t^k, a_t^k, r_t^k, s_{t+1}^k)$  from  $D^k$ 
14:      if episode is not terminal then
15:        Set target  $y_t^k = r_t^k + \gamma \cdot \max_a Q_{\text{target}}(s_{t+1}^k, a; \theta_{\text{target}}^k)$ 
16:      else
17:        Set target  $y_t^k = r_t^k$ 
18:      end if
19:      Update Q-network  $L(\theta^k) = \frac{1}{|B|} \sum_t (Q(s_t^k, a_t^k; \theta^k) - y_t^k)^2$ 
20:      Every  $I$  steps, reset target Q-network:  $\theta_{\text{target}}^k = \theta^k$ 
21:      Every  $I$  steps, decrease  $\epsilon$ 
22:       $R = R + r_t^k$ 
23:      Move to the next state:  $s_t^k = s_{t+1}^k$ 
24:    end for
25:  end for
26: end for

```

availability. To mitigate potential conflicts, the GPU agent is scheduled first due to its critical role in computational speed.

Each agent uses two neural network models: θ^k as the Q-network and θ_{target}^k as the target Q-network. These networks provide stable model training, addressing the moving target problem in RL algorithms. A replay buffer D^k assists the DQN in selecting appropriate actions based on past experiences.

When timestamp t is triggered, the agent determines the action (a_t^k) based on the current state s_t^k , updated by the AIaaS monitoring system. The action a_t^k is selected from the Q-network or randomly with probability ϵ , balancing exploration and exploitation.

After scheduling $J_{i,j}$ from action a_t^k , the environment responds with a new state s_{t+1}^k and reward r_t^k . The iteration history for agent k is recorded in the replay buffer D^k as the tuple $(s_t^k, a_t^k, r_t^k, s_{t+1}^k)$. By sampling transitions from the replay memory, we reduce the correlation between consecutive samples and the risk of overfitting. The minibatch $(s_t^k, a_t^k, r_t^k, s_{t+1}^k)$ from the replay buffer D^k is used to compute the target Q-value y_t^k , which the Q-network approximates. The Bellman equation, with discount factor γ , is used to prioritize immediate rewards over future rewards.

Once the target value y_t^k is determined, the Q-network is updated by minimizing the loss $L(\theta^k)$ between the predicted and target Q-values for each minibatch size B , enhancing decision-making through backpropagation. To stabilize learning, the target Q-network weights are updated every I step to

TABLE IV
STATE DEFINITIONS

	Component	Num of input resolution
Task State	$Z_{i,j}, D_{i,j}, M_{i,j}, B_{i,j}, I_{i,j}$	5
	$T_{Ai,j}$	1
Resource State	GPU	$G^C + mG^E + mnG^F$
	CPU	$C^C + mC^E + mnC^F$
Statistical State	ratios of AI type	3
	remaining task num in task pool	1
	avg. queuing time	1
	avg. execution time	1
Total size:		$6 \times J$ $+G^C + mG^E + mnG^F$ $+C^C + mC^E + mnC^F$ $+6$

match the main Q-network. By decreasing the exploration rate ϵ over time, the algorithm shifts towards exploiting learned knowledge to maximize rewards. The final reward R evaluates the RL solution's performance.

Following RL conventions, we adopt the Markov decision process framework, with states s_t^k , actions a_t^k , and rewards r_t^k defining the decision-making at time t . Below are the definitions of these elements within our approach:

1) **States** (s_t^k): At the central orchestrator level, monitoring supervises three critical components: the task pool, resource status, and statistical analysis. These components serve as sub-states in our tuple, denoted as task states, resource states, and statistical states, as shown in Table IV. These states guide the RL agent's decisions regarding $J_{i,j}$.

Task state (s_t^{task}): Tasks are added to the task pool following FIFO behavior, allowing the RL agent to examine each task's AI information with the monitor's help. The pool contains J tasks when the scheduler is triggered. AI information includes dataset samples $D_{i,j}$, AI model size $M_{i,j}$, processing batch size $B_{i,j}$, and processing iteration $I_{i,j}$, which significantly influence AI task performance [28].

Resource state ($s_t^{resource}$): CPU and GPU states provide more than binary occupation indicators. Our monitoring system calculates expected finish times after brief task execution periods, offering future resource release times to enhance the RL agent's task scheduling decisions.

Statistical state (s_t^{stat}): The stat state provides insights beyond individual task details. It computes metrics such as the current number and ratio of AI types in the queue and average queue waiting and execution times based on prior scheduling outcomes. This historical data helps the RL agent understand request flow and make efficient scheduling decisions.

2) **Actions** (a_t^k): Each agent initially selects an available resource. The RL agent then evaluates state information to make scheduling decisions. It can schedule a task from the queue or pass the current situation to the next agent, retaining resources for future tasks and enhancing efficiency. For instance, if a task $J_{i,j}$ is evaluated by the GPU agent but not scheduled, the task can be passed to the CPU agent for consideration. Furthermore, if a task $J_{i,j}$ is scheduled by the GPU agent, we measure the scheduled time $T_{Si,j}$ and estimate $U_{Ei,j}$ through small execution iterations, which involve running a brief portion of the task to accurately predict its overall processing time without committing extensive resources. The

total data size $rM_{i,j} + sD_{i,j}$ for an AI task moving between data centers accounts for bandwidth L and D , thus considering latency. This expression combines the data required for both the AI model and the dataset into a single total data size. The first part, $rM_{i,j}$, gives the total size of the AI model data, and the second part, $sD_{i,j}$, gives the total size of the dataset. The sum of these two components represents the total amount of data that needs to be transferred, processed, or stored for the task $J_{i,j}$. The departure time from the destination data center can be calculated using the following equations:

$$T_{Di,j} = U_{Ei,j} + T_{Si,j} + \begin{cases} \frac{rM_{i,j} + sD_{i,j}}{L^{GE}} + \frac{rM_{i,j} + sD_{i,j}}{L^{FE}} + D^{GE} + D^{FE} & \text{if } G_{i,j} = 1 \\ \frac{rM_{i,j} + sD_{i,j}}{L^{FE}} + D^{FE} & \text{if } G_{i,j} = 2 \\ 0 & \text{Otherwise} \end{cases}$$

3) **Reward** (r_t^k): To minimize turnaround time, we include it in our reward function: $U_{Ti,j} = T_{Di,j} - T_{Ai,j}$ where $T_{Ai,j}$ is the user's request time, and $T_{Di,j}$ is computed using the earlier formula. RL sums the rewards of tasks in the arrival set to evaluate performance, assigning additional weight to inference tasks due to their typically shorter computation times. Emphasizing penalties for long queue waits for inference tasks promotes efficient processing. With $c_2 > c_1 > 0$, RL prioritizes inference tasks, offering higher rewards and striving to reduce average turnaround times. If inference tasks exceed the Δ^I turnaround time, a penalty Δ^I applies to the reward, normalized to one. The reward function is structured as follows:

$$r_t^k = \begin{cases} \frac{c_1}{T_{Ti,j}} & \text{for } Z_{i,j} \neq 3 \\ \frac{c_2}{T_{Ti,j}} & \text{for } Z_{i,j} = 3, T_{Ti,j} \leq \Delta^I \\ \frac{c_2}{T_{Ti,j}} \times \left(\frac{1}{T_{Ti,j} - \Delta^I + 1} \right) & \text{for } Z_{i,j} = 3, T_{Ti,j} > \Delta^I \end{cases}$$

C. Distributed RL Scheduler

In a multi-data center, multi-tier architecture, capacities differ among fog, edge, and cloud, with the cloud having the most resources and the fog the least. A centralized scheduler typically results in prolonged latency for all tasks due to network topology. To alleviate this, we propose a distributed scheduler, with each data center hosting its own scheduler. This approach considers geographical details and specific data center conditions, coordinating task pools and resource information to minimize latency by distributing scheduling decisions across tiers.

Transforming a centralized scheduler into a distributed one involves coordinating tasks between schedulers and expanding options beyond just GPU and CPU to include other data centers. This vertical scheduling, from fog-edge to edge-cloud, enhances resource availability and promotes efficient allocation across diverse environments.

We propose enhancing our single-data center RL scheduler into a structure with three agents: the GPU agent G , the CPU agent C , and the parent agent P , as shown in Fig. 4. This structure prioritizes GPU and CPU agents,

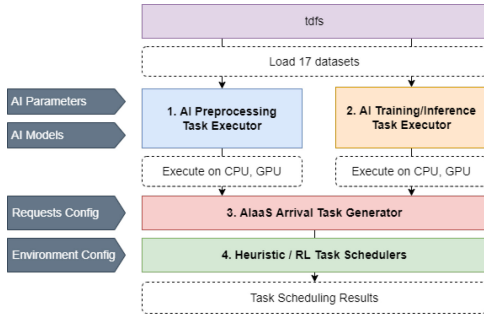


Fig. 5. Simulated workflow of our task scheduling environment.

independent management of resources across data centers and enabling task offloading to higher-tier centers. The differences between these algorithms are highlighted in bold and italic text in Algorithm 2.

V. IMPLEMENTATION

A. Implementation Details

The workflow of our experiments involves obtaining real measurements of AI task execution times on both GPU and CPU independently and simulating AIaaS environments for task scheduling. This process includes four components: AI preprocessing task executor, AI training/inference task executor, AIaaS arrival task generator, and heuristic/RL task schedulers, as shown in Fig. 5.

- We start by producing datasets from tdfs, followed by filtering and cleanup. Unique preprocessing procedures are generated for distinct AI tasks and datasets, executed on both CPU and GPU to measure execution times. This creates diverse task scheduling datasets.
- We construct 17 Deep Neural Network (DNN) models using PyTorch, including popular models such as Squeezenet, Mobilenetv2, and Resnet50. These models are trained on datasets, with execution times measured on CPU and GPU. Adjusting AI parameters creates diverse training and inference task datasets for scheduling.
- In our AIaaS environment simulation, we generate a dataset of incoming tasks by combining preprocessing, training, and inference CPU-GPU execution outcomes. This input dataset encompasses diverse arrival times, quantities, and task distributions, informed by trace data from an operational AIaaS production cluster.
- We implement simulated schedulers using heuristic or RL algorithms. The output includes scheduled tasks, allocated resources, and timestamps. Experimental results are evaluated based on timestamps, providing insights into temporal dynamics and facilitating the refinement of scheduling strategies for improved performance and scalability.

B. Testbed

Our simulation tool, developed using Python, runs on the Ubuntu 22.04.1 LTS system. The hardware setup comprises an Intel Core i7-8700 CPU clocked at 3.20GHz and an NVIDIA GeForce RTX 3060 GPU with 12GB of memory,

supported by CUDA 12.0. All procedures for generating AI tasks and optimizing task scheduling are executed within this environment.

C. Open Sources and Tools

We conducted our experiments using several open-source tools. TensorFlow Datasets (TFDS) provided 17 diverse datasets covering various stages of the ML lifecycle, including caltech101, cifar10, emnist, fashion_mnist, and stanford_dogs. The Alibaba Cluster Trace Program offered real-world AI/ML workload traces, helping configure parameters and analyze resource utilization in AIaaS environments. Additionally, we used PyTorch, a widely adopted ML framework, for model development and experimentation.

VI. RESULTS

This section begins with parameter settings, followed by experimental results addressing two key areas: (1) comparing RL and heuristic approaches in handling AI request rates, CPU-GPU selection, AI task types, and their impact on queuing and execution times; and (2) evaluating centralized vs. distributed RL scheduler models, with a focus on decision latency in inference tasks.

A. Parameter Settings

Table V shows the parameter configurations for our simulation, organized into parameters, values, and descriptions.

Our topology includes two edge data centers per cloud data center and two fog data centers per edge data center, totaling seven data centers. Computation units in cloud, edge, and fog data centers are divided into CPU and GPU units, with quantities sourced from existing testbeds [30], [31], [32].

Network latency involves link bandwidth and propagation delay. Link bandwidth follows 5G network standards: 10 Gbps between fog and edge nodes, and 100 Gbps between edge and cloud nodes. Propagation delay is based on distances between data centers, sourced from existing literature [33].

For latency model calculations, we use a sample size of 24,239 bytes from ImageNet2012 and a register data size of 4 bytes (*float32*), typical for neural networks. An industry report [34] suggests keeping AI inference time under 1.3 seconds for a responsive user experience.

Table VI outlines the parameter settings for our RL agent. The symbol γ denotes the discount rate in our Bellman equation, prioritizing immediate rewards. ϵ governs the ϵ -greedy policy, where the agent starts by exploring actions randomly with a probability of 0.9, gradually decreasing ϵ with a decay rate ρ per episode, until reaching the minimum threshold ϵ_{min} . The learning rate α dictates the magnitude of parameter updates during training. The constants N , B , and I represent parameters for configuring the DQN network and environment.

B. Algorithms: RL vs. Heuristic

In this section, we conduct experiments on a centralized architecture to analyze performance differences due to

TABLE V
BASELINE CONFIGURATION FOR EXPERIMENT

Parameters	Value	Description
m	2	
n	2	
G^C	64	
G^E	16	
G^F	4	
C^C	96	
C^E	24	
C^F	8	
L^{FE}	10	Unit: Gbps
L^{EC}	100	Unit: Gbps
D^{FE}	3.335×10^{-5}	Unit: Seconds
D^{EC}	3.335×10^{-3}	Unit: Seconds
$D_{i,j}$	1, 8, 16, 2048, 4096, 8192, 16384, 32768	Randomized per task
$M_{i,j}$	17 DNN models	Randomized per task
$B_{i,j}$	8, 32, 128	Randomized per task
$I_{i,j}$	1, 20, 50, 100	Randomized per task
r	4	Unit: Byte
s	24239	Unit: Byte

TABLE VI
RL PARAMETERS CONFIGURATION FOR EXPERIMENT

Parameter	Value	Description
γ	0.9	Discount rate
ϵ	0.9	Initial probability of random action
ϵ_{min}	0.1	Minimum probability of random action
ρ	0.95	Decay rate for ϵ
α	0.0005	Learning rate
N	500	Replay memory size
B	32	Minibatch size
I	4000	Update frequency

algorithm variations. For most of the experiments, we use turnaround time (TAT) as the primary performance metric. TAT represents the total time taken for a task from its arrival until its completion, including both queuing and execution times. We compare our RL scheduler against a traditional scheduling algorithm and an existing heuristic method. Since our environment does not support preemptive task execution, we use First-Come-First-Served (FCFS) and Shortest Job First (SJF) algorithms. These algorithms greedily predefine resource allocation by using GPUs, the default in common scheduling configurations. We also use the heuristic approach “TYPE” from [17]. Below are the definitions of those algorithms:

- First-Come-First-Served (FCFS): This default algorithm prioritizes tasks by arrival time, selecting resources with the shortest waiting time, and executing tasks in arrival order.
- Shortest Job First (SJF): This algorithm prioritizes tasks by the shortest execution time. For simulation simplicity, we assume the scheduler knows exact task durations, facilitating SJF implementation.
- Types of AI Tasks (TYPE): From [17], AI tasks are classified as Low-GPU or High-GPU. Low-GPU tasks, typical of preprocessing, use CPUs, while High-GPU tasks, typical of training and inference, use GPUs for parallel acceleration.

1) *Impact of AI Request Rate*: Tenant request rates are varied in simulations, with results shown in Fig. 6, which includes error bars to indicate the variability across multiple

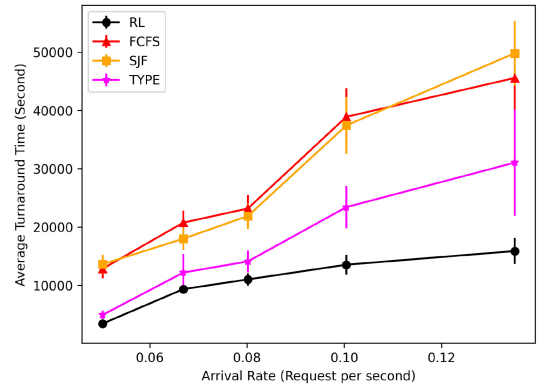


Fig. 6. Average turnaround time for different request rates.

experimental runs. Higher task request rates lead to longer turnaround times for all schedulers due to increased waiting times for resources. The figure shows four lines: black for our centralized RL scheduler and others for heuristic methods—FCFS, SJF, and the TYPE algorithm.

Heuristic methods FCFS and SJF fluctuate across request rates, indicating that singular metrics such as queuing and execution times do not fully represent scheduling objectives. The TYPE algorithm, designed for AI task characteristics, outperforms traditional methods at every request rate. The RL algorithm outperforms all heuristic algorithms, showing greater improvements in turnaround time as the arrival rate increases.

For example, RL reduces turnaround times by over 8000 seconds at a rate of 0.14 and around 1000 seconds at a rate of 0.05 compared to the TYPE algorithm. RL reduces turnaround time by 27.66% compared to TYPE, 57.81% compared to SJF, and 57.28% compared to FCFS.

To further evaluate the effectiveness of our RL-based scheduler beyond turnaround time, Fig. 7 presents throughput analysis across different task arrival rates, measured as the number of completed tasks per 1000 seconds. As arrival rates increase, all methods experience a decline due to resource contention, but RL consistently achieves higher throughput, especially at moderate to high rates. FCFS suffers the most significant drop, while SJF and TYPE perform better but still lag behind RL, which dynamically adjusts task allocation based on system conditions. By balancing workloads and making real-time scheduling decisions, RL maintains a higher number of completed tasks per unit of time, confirming its versatility in improving both turnaround time and throughput in AIaaS environments.

To explain why the RL algorithm outperforms heuristic algorithms, we provide the analysis based on interleaving results in the following sub-sections.

Summary: Higher request rates lead to longer turnaround times, yet RL reduces turnaround time by at least 27.66% compared to heuristic algorithms.

2) *Benefit From CPU-GPU Selection: RL vs. TYPE*: To evaluate the benefits of RL over the TYPE heuristic algorithm, we analyze its overall efficiency in CPU-GPU selection by comparing average execution time, queueing time, TAT, and

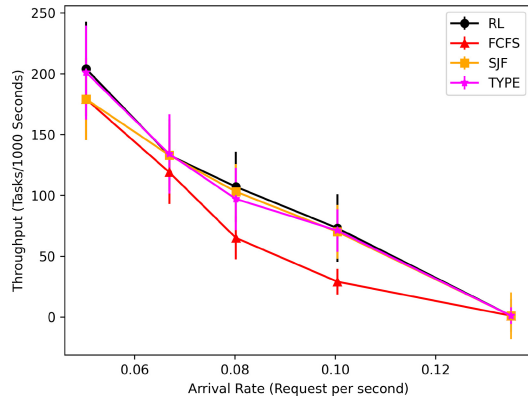


Fig. 7. Throughput for different request rates.

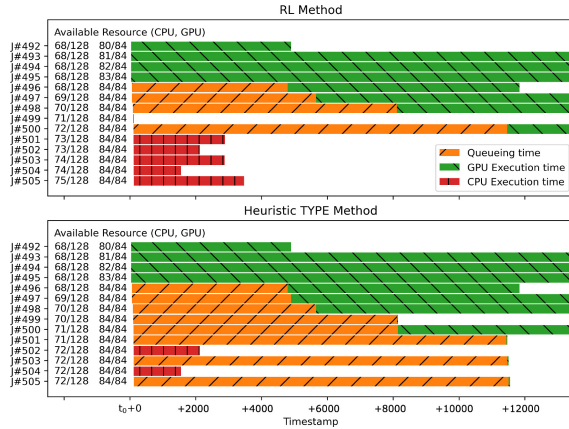


Fig. 8. Case study in task scheduling between RL and TYPE.

task distribution across CPU and GPU. The results confirm that RL significantly reduces queueing times, with CPU queueing time decreasing from 997.77 seconds in TYPE to 605.73 seconds in RL and GPU queueing time reducing from 907.52 seconds in TYPE to 633.34 seconds in RL. By minimizing waiting times, RL improves overall system responsiveness and ensures more efficient resource utilization.

Regarding execution times, RL achieves a lower CPU execution time of 35473.13 seconds compared to 39342.66 seconds in TYPE, demonstrating improved efficiency in CPU task handling. Meanwhile, RL's GPU execution time is 996.28 seconds, slightly higher than 563.75 seconds in TYPE, because RL assigns a larger proportion of tasks to GPUs—72.75% in RL compared to 67.50% in TYPE. Although this increases the total GPU workload, it ensures that more tasks benefit from GPU acceleration while reducing queueing delays, leading to a more balanced execution flow.

In terms of overall turnaround time, RL reduces TAT for CPU tasks by 10.55%, from 40340.43 seconds in TYPE to 36078.86 seconds, and reduces GPU TAT by 10.75%, from 1471.27 seconds in TYPE to 1629.62 seconds. RL's ability to forecast workloads and dynamically adjust task scheduling prevents long-term resource occupation, ensuring that both CPU and GPU availability remain flexible for future workloads. This predictive scheduling capability allows

RL to optimize resource allocation, avoiding bottlenecks and improving execution efficiency.

A specific scenario further highlights RL's effectiveness in CPU-GPU selection, as shown in Fig. 8. The figure shows timestamps on the x-axis and AI tasks on the y-axis, sorted by arrival time. Each task is represented by a horizontal bar, with queueing time in yellow and execution time in green (GPU) or red (CPU). The left column shows CPU and GPU availability, with text colors indicating their status.

Both RL and TYPE algorithms are tested under the same scenario. Initially, all resources are available, and both methods prioritize GPU acceleration for executing tasks. However, as more tasks arrive, resource availability changes dynamically. When task J#496 arrives, GPU resources are fully occupied. The TYPE algorithm follows a static scheduling policy, waiting for GPU availability rather than reallocating tasks dynamically. This results in prolonged queueing times, which are represented by the extended yellow bars in Fig. 8. RL, on the other hand, dynamically offloads some tasks to CPU execution instead of waiting for GPU resources to free up, ensuring continuous task execution and reducing overall delays. While this temporary increase in CPU execution is visible in the figure, the broader results confirm that RL achieves lower CPU execution time overall and better balances GPU utilization across the entire scheduling process.

Summary: RL dynamically selects between CPU and GPU to reduce queueing and execution times, whereas TYPE relies solely on GPU acceleration.

3) Impact of AI Task Types: Preprocessing, Training, and Inference: We examined the RL scheduler's performance from the perspective of AI tasks. Fig. 9 shows the impact of different AI task types on average turnaround time. The x-axis categorizes AI tasks (preprocessing, training, inference), and the y-axis represents the average turnaround time.

The type of AI task significantly affects turnaround time. Inference tasks have shorter execution times, training tasks take the longest, and preprocessing tasks do not benefit from GPU acceleration. This affects the TYPE heuristic algorithm, which uses task type as its scheduling metric.

For preprocessing tasks, all four algorithms show minimal differences, with turnaround times from 0.87x to 1.11x of the RL baseline. The limited benefit of GPU acceleration and the infrequent exhaustion of CPU resources restrict optimization potential. SJF performs better due to the smaller, sequential nature of preprocessing tasks.

Training tasks greatly impact average turnaround time. The RL algorithm achieves the lowest turnaround time, with FCFS and SJF taking nearly twice as long, and the TYPE algorithm's time being 31% longer than the RL baseline. FCFS and SJF's heuristic metrics do not fully address scheduling needs, and the TYPE algorithm lacks dynamic utilization of heterogeneous resources.

For inference tasks, differences among algorithms are smaller but still significant. The RL scheduler outperforms others by 25% to 38% in turnaround times, as it efficiently utilizes CPU resources when GPUs are unavailable, avoiding long queues.

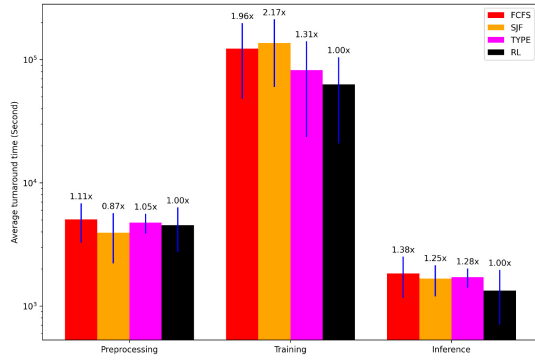


Fig. 9. Average turnaround time by AI task types.

Overall, the RL scheduler excels in reducing average turnaround time, particularly for training and inference tasks. Monitoring task-type distributions can help AIaaS providers optimize performance and resource allocation.

Summary: RL shows limited improvement for preprocessing tasks but significant improvement for training and inference tasks compared to heuristic algorithms.

4) *Impact of the Scheduling Algorithms on Queuing and Execution Times:* Fig. 10 shows the results of heuristic schedulers and our centralized RL scheduler, focusing on queuing and execution times. The x-axis represents each algorithm, while the bar chart illustrates the two scheduling stages.

FCFS and SJF do not consider resource type in their heuristics, resulting in minimal decision and queuing times (491.24 and 644.15 seconds). However, their execution times are prolonged (29,600.78 and 27,693.54 seconds), leading to the worst average turnaround times among the traditional algorithms.

The TYPE algorithm has strict resource requirements, causing a long queuing time of 5530.48 seconds. However, it achieves the shortest execution time (4114.87 seconds) due to GPU acceleration. Despite this, the prolonged queuing time is a significant limitation.

The RL algorithm balances both queuing and execution times, resulting in the shortest turnaround time with 1145.08 seconds of queuing and 4927.58 seconds of execution.

The results show that both queuing and execution times affect scheduling performance. AIaaS providers should consider resource availability to optimize task scheduling and improve performance.

Summary: RL optimizes task scheduling by reducing the queuing and execution times, whereas heuristic algorithms address only one aspect.

C. Centralized RL vs. Distributed RL

We compare the performance of our centralized RL scheduler with a distributed RL scheduler under high load and heterogeneous AI tasks. We examine various AI task types, the offloaded percentage in a distributed architecture, and the latency of both schedulers.

Both schedulers operate within our distributed network architecture, with equal access to fog, edge, and cloud data centers, and identical computational resources and network

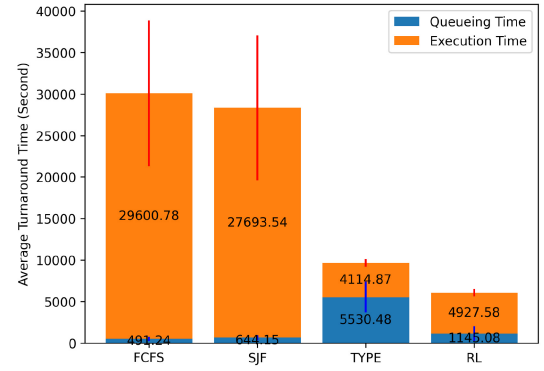


Fig. 10. Average turnaround time by the algorithms.

latency. The primary difference is in the number and location of the schedulers: the centralized scheduler is in the cloud data center, while the distributed configuration has one scheduler in the cloud, m in edge data centers, and mn in fog data centers. We trained and tested both schedulers using identical task requests to compare their outcomes.

The time complexity of the Centralized RL Scheduler (Algorithm 1) is determined by the number of episodes M , the maximum time steps per episode $T_{\max}$, the minibatch size B , and the computational cost of a forward and backward pass through the Q-network F . At each time step, each of the two agents (GPU and CPU) selects an action via an ϵ -greedy policy $O(F)$, stores transitions $O(1)$, samples a minibatch for training $O(B)$, computes the target Q-value $O(F)$, and updates the network via backpropagation $O(B \cdot F)$. This results in an overall time complexity of $O(M \cdot T_{\max} \cdot B \cdot F)$. The space complexity is dictated by the replay memory size N and the number of parameters in the Q-network P , leading to $O(N + P)$.

The Distributed RL Scheduler (Algorithm 2) follows a similar computational process but introduces an additional agent, increasing computational overhead. Each of the three agents independently selects an action, samples a minibatch, computes target Q-values, and updates the Q-network, incurring the same per-agent cost of $O(B \cdot F)$. The additional state merging operation adds a minor $O(1)$ cost. Consequently, the overall time complexity remains $O(M \cdot T_{\max} \cdot B \cdot F)$, but with a higher constant factor due to the extra agent. Similarly, the space complexity remains $O(N + P)$, but unlike the centralized approach, which maintains a single replay memory and Q-network, the distributed version requires multiple instances—one per agent—leading to a higher absolute memory requirement.

In summary, both the centralized and distributed RL schedulers share the same time complexity of $O(M \cdot T_{\max} \cdot B \cdot F)$, but the distributed scheduler has a higher constant factor due to an additional agent. Similarly, both have the same space complexity of $O(N + P)$, but the distributed version requires more memory since each of its three agents maintains an independent replay buffer of size N and a Q-network with P parameters. This highlights the trade-off between scalability and increased computational and memory costs in the distributed approach.

1) *Performance by AI Task Types*: Fig. 11 shows the average turnaround time for centralized and distributed schedulers across different AI task types. The red line indicates the standard error mean (SEM), showing the distribution of the results. Overall, the centralized scheduler performs better when it has a comprehensive view of all computational resources. However, breaking down the results by preprocessing, training, and inference tasks reveals more nuanced insights.

For preprocessing tasks, both schedulers perform similarly because these tasks primarily use CPU resources, which do not face significant bottlenecks. The performance of CPU and GPU units in preprocessing tasks is nearly equivalent.

Training tasks, which have long execution times, heavily influence the overall performance. The centralized scheduler, with its full visibility of resources, optimizes usage and reduces turnaround time for training tasks by 29.05%. However, it struggles with scheduling short inference tasks when long tasks occupy resources in the data center.

In contrast, the distributed scheduler learns to offload long training tasks to the cloud, freeing edge and fog resources. This approach reduces inference task turnaround time by 89.07% compared to the centralized scheduler, with the SEM showing consistent completion times.

The distributed RL scheduler improves scalability through localized decision-making but faces limits as system size increases. Communication overhead between nodes can delay state updates and task synchronization, especially in dense networks. Additionally, expanding state and action spaces with more nodes increases computational complexity, potentially impacting the scheduler's responsiveness. Balancing workloads across heterogeneous nodes also becomes challenging as resource disparities grow.

Furthermore, some challenges—such as adapting to dynamic workloads with unpredictable traffic patterns and synchronizing states across geographically dispersed nodes—are present in both centralized and distributed RL schedulers. However, in a distributed RL setting, these issues become more pronounced due to the need for frequent inter-node communication and coordination delays in environments with low bandwidth or high latency. Similarly, while task profile variations and energy constraints in edge or fog tiers affect both approaches, they pose greater difficulties in a distributed RL framework, where decentralized agents operate under varying computational capabilities and network conditions.

Despite its challenges, distributed RL offers critical benefits. Decentralized decision-making reduces bottlenecks and improves responsiveness, especially for short-duration tasks like inference. Localized scheduling adapts to resource availability in real time, minimizing delays. Moreover, distributed RL scales effectively by leveraging hierarchical or federated structures, making it ideal for heterogeneous environments with diverse resource and workload demands.

Summary: Centralized scheduling is suitable for long-duration tasks such as training, while distributed scheduling is better for short-duration tasks such as inference.

Here's the revised section focusing solely on decision time:

2) *Decision Latency on Inference Tasks*: Fig. 12 illustrates the decision latency in our inference test cases. The decision

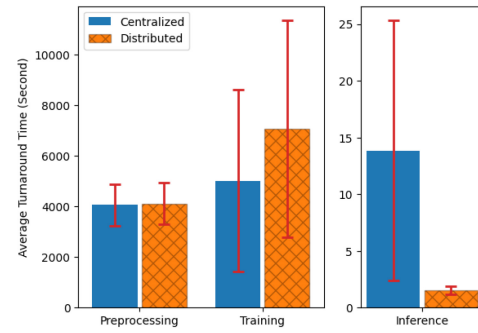


Fig. 11. Average turnaround time of two schedulers by AI task types.

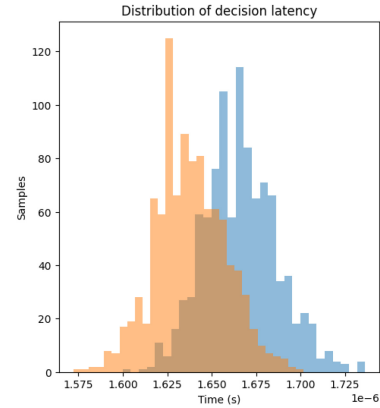


Fig. 12. Decision latency from inference tasks.

time, which is approximately 0.0016 milliseconds for both the centralized and distributed schedulers, is negligible and does not hinder inference tasks that require responsiveness within 1.3 seconds. The distributed scheduler shows slightly better decision times due to its smaller neural network architecture.

Given the quick decision-making process, our RL scheduler efficiently learns optimal scheduling decisions over thousands of epochs without introducing overhead for short inference tasks. With the shortest inference tasks taking around 250 milliseconds, the RL scheduler's decision time occupies only a minimal fraction of the overall turnaround time, highlighting the scheduler's efficiency in handling inference tasks.

Summary: The RL scheduler's decision time is extremely short, ensuring that it contributes minimally to the overall delay in short inference tasks.

D. Fine-Tuning RL Parameters: Impact and Observation

In configuring our RL parameters, we followed the methodology in [35]. Fig. 13 shows the influence of hyperparameter selection on the success of the deep Q-learning algorithm. Each plot represents a training run with different parameter settings, depicting the average reward across epochs.

We focus on three crucial RL parameters: learning rate (lr), decay rate (dr), and γ . The learning rate dictates the size of optimization steps. A higher lr can speed up convergence but risk overshooting, while a lower lr may improve convergence

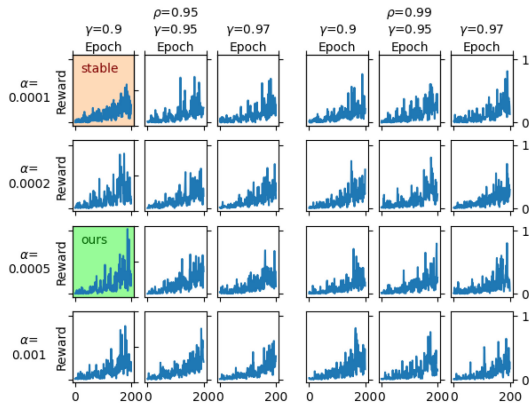


Fig. 13. Reward variation across different RL parameter settings.

but prolong training. With $\gamma = 0.9$ and $\rho = 0.95$, a learning rate of 0.0001 consistently boosts rewards, while 0.001 achieves a superior solution with a broader distribution.

The decay rate indicates how quickly the exploration rate ϵ decreases during training, balancing exploration and exploitation. We found that the decay rate had little impact on our dynamic environment.

The discount factor γ measures the significance of future rewards. A higher γ suggests prioritizing future rewards, complicating convergence. We observed higher γ values made convergence more challenging.

Dynamic traffic impacts RL scheduler performance, influencing the learning rate, decay rate, and discount factor. Parameters ρ and γ with high values may hinder convergence. The combination of a learning rate of 0.0005, γ of 0.9, and a decay rate of 0.95 yielded the highest reward. For stable convergence, a learning rate of 0.0001 can be used. AIaaS providers should consider request trace distribution and user intent for effective RL parameter fine-tuning.

Summary: Dynamic AI task requests in AIaaS environments significantly influence RL performance. Effective fine-tuning of RL parameters should consider the patterns of incoming requests and user behavior to optimize scheduling outcomes.

VII. CONCLUSION AND FUTURE WORKS

We proposed the UXP-RL algorithm, which is an adaptive RL approach to address task scheduling in a complex environment characterized by diverse AI tasks (preprocessing, training, inference), heterogeneous resources (GPU, CPU), and a distributed network architecture (cloud, edge, fog). We evaluated task scheduling outcomes from two perspectives.

Firstly, we compared our UXP-RL algorithm with heuristic approaches, analyzing request intervals, turnaround time, training procedures, and resource usage. Secondly, we evaluated centralized and distributed scheduler deployments.

Based on our experiments, we offer these recommendations for AIaaS providers in designing their AI task schedulers:

- RL algorithms significantly outperform heuristic methods in task scheduling, reducing average turnaround time by 27.66% to 57.81%. Unlike heuristic algorithms, which typically address only one aspect of scheduling, RL

optimizes both queuing and execution times. AIaaS providers are encouraged to adopt RL methods, as they not only deliver superior results but also offer substantial improvements over time.

- Centralized scheduling is more suitable for long-duration tasks, reducing average turnaround time by 29.05% compared to distributed scheduling. Conversely, distributed scheduling excels with shorter tasks, reducing average turnaround time by 89.07%. Providers should consider tenant request traces and operational scale when choosing between centralized or distributed schedulers.

Despite its advantages, our RL-based scheduler has limitations. Its scalability in large-scale AIaaS environments with thousands of concurrent tasks remains to be fully tested. The high computational overhead during training is another challenge, where optimizations like transfer learning could improve efficiency. Additionally, the model assumes stable resource availability, though real-world AIaaS environments often face fluctuations.

Future work will focus on adaptive mechanisms to handle dynamic system conditions more effectively. Lastly, security and fault tolerance considerations have not been a primary focus of this study. Future research will explore incorporating security-aware scheduling policies and fault tolerance mechanisms to ensure the reliability of AIaaS platforms. Addressing these limitations will pave the way for a more scalable, efficient, and resilient AI-driven scheduling framework.

REFERENCES

- [1] E. Paraskevoulakou and D. Kyriazis, "ML-FaaS: Toward exploiting the serverless paradigm to facilitate machine learning functions as a service," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 3, pp. 2110–2123, Sep. 2023.
- [2] M. M. Taye, "Understanding of machine learning with deep learning: Architectures, workflow, applications and future directions," *Computers*, vol. 12, no. 5, p. 91, 2023. [Online]. Available: <https://www.mdpi.com/2073-431X/12/5/91>
- [3] S. Parsaeefard, I. Tabrizian, and A. Leon-Garcia, "Artificial intelligence as a service (AI-aaS) on software-defined infrastructure," in *Proc. IEEE Conf. Stand. Commun. Netw. (CSCN)*, 2019, pp. 1–7.
- [4] Y.-D. Lin, Y.-C. Lai, D. Sudyana, and R.-H. Hwang, "Artificial intelligence for Internet of Things as a service: Small or big data, private or public model, Centralized or federated learning?" *Computer*, vol. 56, no. 12, pp. 65–79, Dec. 2023.
- [5] M. Amaral, J. Polo, D. Carrera, S. Seelam, and M. Steinder, "Topology-aware GPU scheduling for learning workloads in cloud environments," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, 2017, pp. 1–12.
- [6] F. Filippini, M. Lattuada, M. Ciavotta, A. Jahani, D. Ardagna, and E. Amaldi, "A path relinking method for the joint online scheduling and capacity allocation of DL training workloads in GPU as a service systems," *IEEE Trans. Services Comput.*, vol. 16, no. 3, pp. 1630–1646, May/Jun. 2023.
- [7] Y. Bao, Y. Peng, and C. Wu, "Deep learning-based job placement in distributed machine learning clusters," in *Proc. IEEE Conf. Comput. Commun.*, 2019, pp. 505–513.
- [8] Y. Xiang, B. Shi, and C. Zhang, "Proteus: Distributed machine learning task scheduling based on Lyapunov optimization," in *Proc. IEEE Int. Conf. Joint Cloud Comput. (JCC)*, 2021, pp. 9–15.
- [9] F. Filippini et al., "ANDREAS: Artificial intelligence training scheduler for accelerated resource clusters," in *Proc. 8th Int. Conf. Future Internet Things Cloud (FiCloud)*, 2021, pp. 388–393.
- [10] Y. Luan, X. Chen, H. Zhao, Z. Yang, and Y. Dai, "SCHED²: Scheduling deep learning training via deep reinforcement learning," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, 2019, pp. 1–7.

- [11] W. Xiao et al., "Gandiva: Introspective cluster scheduling for deep learning," in *Proc. 13th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, 2018, pp. 595–610.
- [12] X. Wu, H. Xu, and Y. Wang, "Irina: Accelerating DNN inference with efficient online scheduling," in *Proc. 4th Asia-Pacific Workshop Netw.*, 2020, pp. 36–43.
- [13] C. Hazott and D. Grosse, "DSA monitoring framework for HW/SW partitioning of application kernels leveraging VPs," in *Proc. Design Verif. Conf. Exhibit. Eur.*, 2023, pp. 34–41.
- [14] Y. Xiao, S. Nazarian, and P. Bogdan, "Self-Optimizing and self-programming computing systems: A combined compiler, complex networks, and machine learning approach," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 6, pp. 1416–1427, Jun. 2019.
- [15] Y. Xiao et al., "End-to-end programmable computing systems," *Commun. Eng.*, vol. 2, no. 1, p. 84, Nov. 2023.
- [16] Y. Xiao, S. Nazarian, and P. Bogdan, "Plasticity-on-chip design: Exploiting self-similarity for data communications," *IEEE Trans. Comput.*, vol. 70, no. 6, pp. 950–962, Jun. 2021.
- [17] Q. Weng et al., "MLaaS in the wild: Workload analysis and scheduling in large-scale heterogeneous GPU clusters," in *Proc. 19th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2022, pp. 945–960.
- [18] P. Gong, Y. Ma, C. Li, X. Ma, and S. H. Noh, "Understand data preprocessing for effective end-to-end training of deep neural networks," 2023, *arXiv:2304.08925*.
- [19] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.
- [20] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
- [21] H. Zhang and T. Yu, "Taxonomy of reinforcement learning algorithms," in *Deep Reinforcement Learning: Fundamentals, Research and Applications*. Singapore: Springer, 2020, pp. 125–133.
- [22] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [23] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 30, 2016, pp. 2094–2100.
- [24] M. G. Bellemare, W. Dabney, and R. Munos, "A distributional perspective on reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 449–458.
- [25] S. Kekki et al., "MEC in 5G networks," ETSI, Sophia Antipolis, France, White Paper, 2018.
- [26] P.-H. Kuo et al., "An integrated edge and fog system for future communication networks," in *Proc. IEEE Wireless Commun. Netw. Conf. Workshops (WCNCW)*, 2018, pp. 338–343.
- [27] D. H. Abdulazeez and S. K. Askar, "Offloading mechanisms based on reinforcement learning and deep learning algorithms in the fog computing environment," *IEEE Access*, vol. 11, pp. 12555–12586, 2023.
- [28] J. Sevilla, L. Heim, M. Hobbhahn, T. Besiroglu, A. Ho, and P. Villalobos, "Estimating training compute of deep learning models." Epoch. 2022. [Online]. Available: <https://epoch.ai/blog/estimating-training-compute>
- [29] J. Gu et al., "Tiresias: A GPU cluster manager for distributed deep learning," in *Proc. 16th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2019, pp. 485–500.
- [30] D. Kimovski, R. Mathá, J. Hammer, N. Mehran, H. Hellwagner, and R. Prodan, "Cloud, fog, or edge: Where to compute?" *IEEE Internet Comput.*, vol. 25, no. 4, pp. 30–36, Jul./Aug. 2021.
- [31] V. Tomer and S. Sharma, "Experimenting an edge-cloud computing model on the GPULab Fed4Fire testbed," in *Proc. IEEE Int. Symp. Local Metrop. Area Netw. (LANMAN)*, 2022, pp. 1–2.
- [32] W. Xiao et al., "AntMan: Dynamic scaling on GPU clusters for deep learning," in *Proc. 14th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, 2020, pp. 533–548.
- [33] Y.-C. Lai et al., "Task assignment and capacity allocation for ML-based intrusion detection as a service in a multi-tier architecture," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 1, pp. 672–683, Mar. 2022.
- [34] D. An. "Find out how you stack up to new industry benchmarks for mobile page speed." Think Google-Mobile. 2018. [Online]. Available: <https://www.thinkwithgoogle.com/intl/en-emea/marketing-strategies/app-and-mobile/find-out-how-you-stack-new-industry-benchmarks-mobile-page-speed/>
- [35] N. Sprague, "Parameter selection for the deep Q-learning algorithm," in *Proc. Multidiscip. Conf. Reinforce. Learn. Decision Making (RLDM)*, 2015, p. 24.



Ying-Dar Lin (Fellow, IEEE) received the Ph.D. degree in computer science from the University of California at Los Angeles in 1993. He is a Chair Professor of Computer Science with National Yang Ming Chiao Tung University, Taiwan. He was a Visiting Scholar with Cisco Systems, San Jose, from 2007 to 2008, the CEO of Telecom Technology Center, Taiwan, from 2010 to 2011, and the Vice President of the National Applied Research Labs, Taiwan, from 2017 to 2018. He cofounded L7 Networks Inc. in 2002 and O'Prueba Inc. in 2018. His research interests include cybersecurity, wireless communications, network softwarization, and machine learning for communications. He has served or is serving on the editorial boards of several IEEE journals and magazines, including the Editor-in-Chief of IEEE COMMUNICATIONS SURVEYS AND TUTORIALS from 2017 to 2020.



Yin-Tao Ling received the M.S. degree in computer science and engineering from National Yang Ming Chiao Tung University, Hsinchu, Taiwan, in 2024. He is currently an Engineer at a hi-tech company, specializing in innovative technology solutions. His research interests include wireless networks, embedded systems, and cloud computing.



Yuan-Cheng Lai received the Ph.D. degree from the Department of Computer and Information Science, National Chiao Tung University in 1997. He joined the faculty of the Department of Information Management, National Taiwan University of Science and Technology in August 2001, and has been a Distinguished Professor since June 2012. His research interests include performance analysis, software-defined networking, wireless networks, and IoT security.



Didik Sudyana received the Ph.D. degree from the Department of Electrical Engineering and Computer Science, National Yang Ming Chiao Tung University in 2024. He is currently an Assistant Professor with the Computer and Network Center, National Cheng Kung University, Taiwan. His research interests include cybersecurity, machine learning, and network design and optimization.